Лабораторная работа № 3
«Построение нейросетевого классификатора для идентификации типа подстилающей поверхности при движении мобильного робота в условиях неоднородной среды»

***Цель работы*:**

- Приобретение практических навыков в решении одной из актуальных задач в построении интеллектуальной СУ автономной робототехнической системы, функционирующей в условиях физически неоднородной среды;
- Практическое изучение особенностей и способов решения задачи несбалансированной классификации, а также расширение инструментария разработчика в части работы с методами машинного обучения;
- Развитие исследовательских навыков работы с методом машинного обучения на примере прикладной задачи и реальной выборке данных;
- Развитие навыков программирования систем вычислительного интеллекта на Python.

Описание подмножеств переменных:

$\{N\} = \{N_1, N_2, N_3\}$ – показания энкодеров: обороты 3-х двигателей;

$\{\omega\} = \{\omega_1, \omega_2, \omega_3\}$ – реальные скорости оборотов 3-х двигателей;

$\{I\} = \{I_1, I_2, I_3\}$ – токи потребления 3-х двигателей;

$\{g\} = \{g_x, g_y, g_z\}$ – показания гироскопа: угловая скорость по 3-м осям;

$\{a\} = \{a_x, a_y, a_z\}$ – ускорение по 3-м осям.

Для классификации подстилающей поверхности могут использоваться как непосредственные данные измерений бортовой сенсорной системы (например, показания датчиков тока двигателей могут служить информацией о трудоемкости перемещения по данной поверхности – см. множество $\mathbf{\{V_1\}}$), так и интегральные величины (см. множества $\mathbf{\{V_2\}}$ **и** $\mathbf{\{V_3\}}$)

$\mathbf{\{V_1\}} = \{ \{\mathbf{N_1}, \mathbf{N_2}, \mathbf{N_3}\}, \{\boldsymbol{\omega_1}, \boldsymbol{\omega_2}, \boldsymbol{\omega_3}\}, \{\mathbf{I_1}, \mathbf{I_2}, \mathbf{I_3}\}, \{\mathbf{g_x}, \mathbf{g_y}, \mathbf{g_z}\}, \{\mathbf{a_x}, \mathbf{a_y}, \mathbf{a_z}\} \}$ – значения переменных, полученные непосредственно сенсорной системой робота;

$\mathbf{\{V_2\}} = \{\mathbf{V_x}, \mathbf{V_y}, \boldsymbol{\Omega}, \mathbf{I_x}, \mathbf{I_y}, \mathbf{I_\varphi}, \mathbf{I_\Sigma}\}$ – множество абсолютных параметров, полученных путем математических преобразований непосредственно измеренных переменных;

$\mathbf{\{V_3\}} = \{\mathbf{T_x}, \mathbf{T_y}, \mathbf{T_\varphi}, \mathbf{T_z}\}$ – множество относительных параметров, полученных путем математических преобразований подмножеств $\{V_1\}$ и $\{V_2\}$:

$$T_x = \frac{V_x}{I_x}; \quad T_y = \frac{V_y}{I_y}; \quad T_\varphi = \frac{\Omega}{I_\varphi}; \quad T_z = \frac{g_z}{I_\varphi}$$

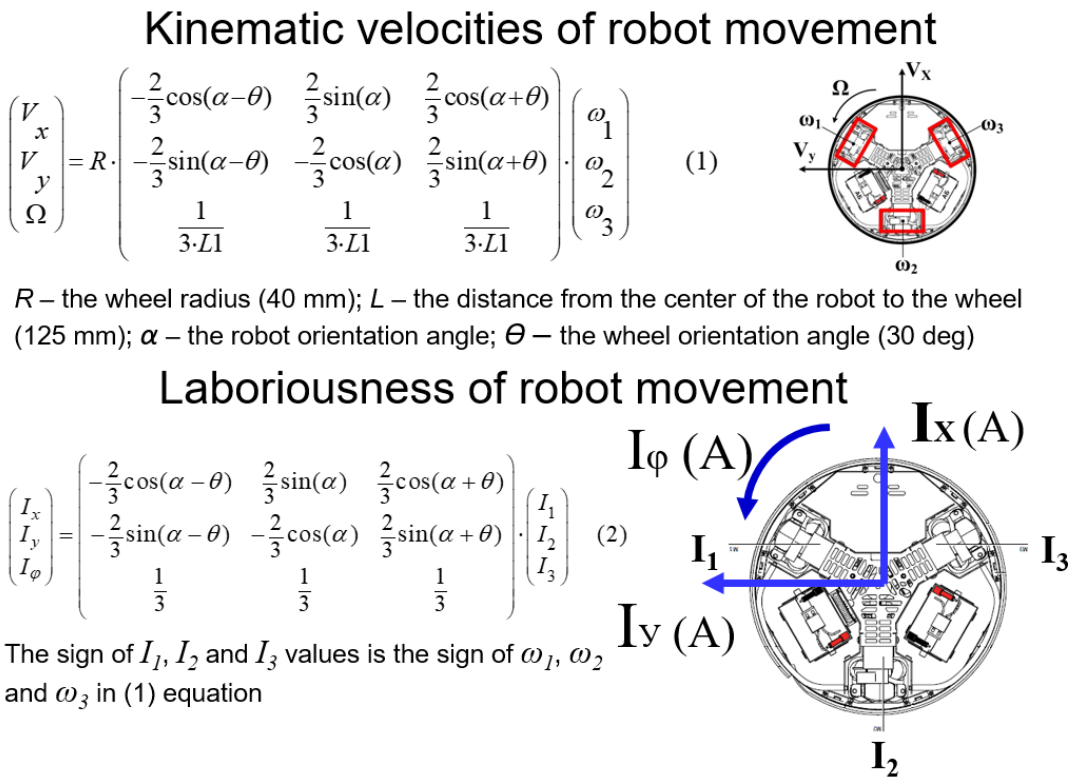

Рисунок 1 – Кинематические преобразования для получения интегральных величин, характеризующих скорость и трудоемкость перемещения робота по поверхности

Задача работы: Построить нейросетевой классификатор (**в соответствии с вариантом задания**), протестированный и более-менее устойчивый к новым данным (самостоятельно осуществить проверку на выборке C), а также приобрести опыт и понимание работы нейросетевого алгоритма в части влияния различных его параметров (числа слоев, нейронов в слое, числа итераций, функции активации, способа предобработки данных и т.п.) на качество получаемой модели.

! Примечание: Данная работа носит преимущественно исследовательский характер и направлена на выработку всесторонних компетенций работы с нейросетевым алгоритмом, касаемо и понимания влияния гиперпараметров самого алгоритма, и способов предобработки данных, и физики задачи на качество получаемых результатов. Проверка качества моделей осуществляется в том числе и на выборке Заказчика – «D», которая имеется у преподавателя лекций курса и за проверку на которой, после защиты лабораторной работы, будут начисляться отдельные баллы).

Краткие рекомендации по выполнению Практикума #2

Для выполнения работы ***рекомендуется установить пакет Anaconda 3*** с официального сайта (ПО класса «Open Source») – https://www.anaconda.com/download/ с учётом типа операционной системы (Windows, Linux), её версии и разрядности (64- или 32-битная).

Практикум рекомендуется выполнять в Jupyter Notebook (входит в пакет Anaconda 3).

После установки пакета Anaconda 3 необходимо установить библиотеку imblearn:

В меню пуск необходимо открыть Anaconda Promt и набрать команду:

pip install imblearn

После нажатия на «Enter» библиотека установится.

<u>Основные этапы выполнения практикума</u>:

1. Для выполнения практикума рекомендуется использовать полносвязную нейронную сеть прямого распространения – MLPClassifier из библиотеки машинного обучения sklearn (см. описание алгоритма на странице библиотеки: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html).

   Допускается использовать другие библиотеки машинного обучения, если у Вас есть опыт работы с ними, однако, алгоритм должен остаться непременно тем, который задан в данном задании – полносвязная нейронная сеть прямого распространения.

   Далее приведена инструкция из предположения использования библиотеки sklearn для реализации нейросетевого классификатора.

2. Провести небольшую предварительную серию экспериментов с параметрами “hidden_layer_sizes”, “activation”, “solver” и “max_iter”, **добиваясь наиболее близких между собой значений критерия кросс-валидации (перекрестного критерия ошибки) на обучающей выборке (Data Set_Train)** для того, чтобы избежать переобучения сети и правильно подобрать параметры алгоритма. При этом сами значения кросс-валидации должны быть не только устойчивыми (близкими к друг другу), но и быть как можно меньше (т.к. представляют собой значения ошибок на тестовых подвыборках).
   С физическим смыслом критерия кросс-валидации можно ознакомиться на странице:
   https://scikit-learn.org/stable/modules/cross_validation.html

   Для вычисления критерия кросс-валидации (cross_val_score) используется модуль ***model_selection*** библиотеки Sklearn (см. страницу https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.cross_val_score.html).

   Параметр “cv”, определяющей число частей, на которые делится выборка, задается на усмотрение разработчика (рекомендуется немного поэкспериментировать с различными значениями параметра “cv” в диапазоне: 3 или 4).

   Также в процессе данных предварительных экспериментов добиваемся увеличения значений *Accuracy (процент угадываний нулей и единиц, т.е. значений любого класса)* и *F1-score* (метрика оценивающая процент угадываний своего класса в сочетании с неперепутыванием его с другими классами).
   Метрика *Accuracy* является очень простой и понятной, однако плохо подходит для несбалансированных выборок (когда количество примеров искомого класса, под который настраивается классификатор, существенно меньше числа примеров других классов), поскольку при хорошем угадывании значений несвоего класса (в данном случае – нулей) значение *Accuracy* уже будет высоким, даже несмотря на то, что свой класс классификатор прогнозирует плохо. Для таких несбалансированных случаев можно использовать *F-меру*, которая является гармоническим средним между точностью и полнотой, она стремится к нулю, если точность или полнота стремится к нулю.

$$F = 2\frac{\text{точность} \times \text{полнота}}{\text{точность} + \text{полнота}}$$

$$\text{точность} = \frac{N_{\text{ип}}}{N_{\text{ип}} + N_{\text{лп}}}$$

$$\text{полнота} = \frac{N_{\text{ип}}}{N_{\text{ип}} + N_{\text{ло}}},$$

где $N_{ИП}$ – количество истинно положительных результатов; $N_{ЛП}$ – количество ложно положительных результатов; $N_{ЛО}$ – количество ложно отрицательных результатов.

С физическим смыслом F-меры и тем, как пользоваться метрикой f1_score, можно ознакомиться на странице документации:
https://scikit-learn.org/stable/modules/generated/sklearn.metrics.f1_score.html

! Примечание: данный этап служит <u>для предварительного знакомства</u> с настройками гиперпараметров алгоритма нейросети и их примерным влиянием на качество получаемого результата. С одной стороны Этап носит ознакомительный характер и поэтому не нужно тратить слишком много времени на его выполнение, непосредственно само исследование в рамках практикума начинается с пункта 3. С другой стороны, данный этап может оказать существенную помощь в дальнейших исследованиях (особенно, если у Вас нет опыта выполнения подобных практикумов), т.к. позволяет более-менее определиться от каких настроек необходимо отталкиваться при проведении дальнейших исследований, поэтому не рекомендуется его пропускать.

3. **<u>Необходимо добиться</u>** в режиме работы «на результат» как можно более высокой точности прогнозирования выходной величины (по критерию F1, а также по устойчивости кросс-валидации) в зависимости от следующих параметров:
   - ***число нейронов*** – по сути это число фильтров нелинейной обработки информации. С одной стороны, их увеличение может помочь обучению модели, т.к. из большего числа фильтров-обработчиков информации, возможно, более легче собрать зависимость. С другой стороны, увеличение числа нейронов существенно увеличивает число связей в сети, поскольку структура сети полносвязная и каждый нейрон следующего слоя связан с каждым нейроном предыдущего слоя (т.е. при 2-х слоях со 100 нейронами число связей будет 10000). А, следовательно, необходимо будет найти больше параметров в процессе обучения (т.е. больше весовых коэффициентов связей между нейронами), что может вступать в противоречие с имеющимся объемом обучающей выборки данных. Желательно, чтобы число примеров выборки было как минимум на порядок больше числа связей в сети (однако, это не означает, что, если соотношение будет в обратную сторону, сеть не сможет обучиться).
     На практике для классической полносвязной нейросети характерны как малые емкости порядка [3 нейрона × 3 слоя], так и большие ёмкости порядка [100 нейронов × 6 слоёв] – конкретные значения емкостей задаются на ваше усмотрение, исходя из опыта, полученного в процессе выполнения пункта 2 и других практикумов. Из Вашего предварительного опыта по п. 2 Вы можете самостоятельно выбрать какую изначально выставить емкость сети.
   - ***число слоев*** – существенно увеличивает сложность модели (т.к. нелинейная функция от взвешенной суммы сигналов на предыдущем слое подставляется в нелинейную функцию текущего слоя), что может помочь быстрее приблизиться к описанию сложных закономерностей в данных. С другой стороны, можно переусложнить модель и она будет хорошо аппроксимировать обучающую выборку, но потеряет прогнозирующую способность (это можно будет отследить по критерию кросс-валидации, т.к. стабильность качества модели будет нарушена).
   - ***функция активации нейронов*** – см. доступные функции активации на странице документации https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html

Некоторые из функций активации (например, logistic или tanh) могут более соответствовать характеру задачи классификации, чем остальные. Однако, некоторые функции активации не очень хорошо работают в паре с определенными алгоритмами обучения (параметр «solver»), поэтому в данной задаче необходимо подбирать пары activation-solver для получения наиболее качественной модели.

- ***алгоритм обучения нейронной сети*** – см. доступные алгоритмы на странице документации https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html
- ***число итераций обучения сети*** – с одной стороны, числа итераций может быть недостаточно, и тогда сеть не будет иметь хорошую прогнозирующую способность. С другой стороны, при большом числе итераций сеть стремиться выучить все примеры выборки данных, в результате чего она слишком подстраивается под конкретные примеры и теряет прогнозирующую способность (т.е. в этом случае сеть просто аппроксимирует обучающую выборку данных, а не вскрывает закономерность между входом и выходом).
- ***random_state*** – настройка генератора случайных чисел, которая влияет на генерируемое случайное множество чисел. Начальные веса связей в нейросети задаются случайно и всё дальнейшее обучение (т.е. настройка этих весов) отталкивается от этих начальных значений, поэтому эта настройка также влияет на качество обучения сети.

4. Зафиксировать лучший полученный результат по п.3 – выставить в блокноте результат лучшего найденного результата обучения (график и значения критериев понадобятся для отчёта). Добавить в блокноте новое поле ниже, выбрать вместо «Code», режим «Markdown» и написать заголовок:

   # Обучение сети на сортированных данных

   После нажатия на Run появится заголовок раздела.

5. Провести эксперимент по п.3 с сортированными данными. Для этого, можно скопировать предыдущий код в новые ячейки блокнота (ниже) и поменять загрузку датасета в функции read_excel на ‘Data Set_Train (Sort).xlsx’.

6. Зафиксировать лучший полученный результат по п.5 – выставить в блокноте результат лучшего найденного результата обучения (график и значения критериев понадобятся для отчёта). Добавить в блокноте новое поле ниже, выбрать вместо «Code», режим «Markdown» и написать заголовок:

   # Обучение сети на нормализованных данных

   После нажатия на Run появится заголовок раздела.

7. Поскольку все датчики имеют различные диапазоны измерений, то логично их привести к одному диапазону (например, [0,1]) для того, чтобы их вклад в функцию нейрона был одинаковым. Для нормализации данных можно воспользоваться функцией MinMaxScaler библиотеки Sklearn (https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html).
   Для этого необходимо добавить импорт из библиотеки и вызвать сам нормализатор:

```
from sklearn.preprocessing import MinMaxScaler

Data = pd.read_excel('Data_Set_(A+B).xlsx')
// Out = ... – в соответствии с вариантом задания
// X = Data.drop(['Type', ... – в соответствии с вариантом], axis=1)

scaler = MinMaxScaler() # Инициализация нормализатора
X_Norm = scaler.fit_transform(X) # Нормализация и запись результата в X_Norm
```

```
...

ANN.fit(X_Norm, Y)

...
```

! Обратите внимание, что после получения нормализованных значений входных переменных в матрице X_Norm, последняя используется в обучении сети вместо X. Более того, далее она используется везде в рамках данного эксперимента (см. п.8) – и в ANN.predict, и в функции cross_val_score (необходимо сделать такую подстановку самостоятельно).

8. Провести эксперимент по п.3 с нормализованными данными. При этом, в качестве выборки данных, которая загружается в переменную Data из Excel, необходимо выбрать ту (с сортировкой или без сортировки), на которой в предыдущих экспериментах результат получился лучше по кросс-валидации и критерию F1.

9. Зафиксировать лучший полученный результат по п.8 – выставить в блокноте результат лучшего найденного результата обучения (график и значения критериев понадобятся для отчёта). Добавить в блокноте новое поле ниже, выбрать вместо «Code», режим «Markdown» и написать заголовок:

   # Обучение сети на сбалансированных данных

   После нажатия на Run появится заголовок раздела.

10. В данной выборке число примеров единичного класса (которому обучается классификатор и учится отделять его от примеров нулевого класса) в процентном отношении существенно меньше числа примеров нулевого класса, поэтому нейросеть при обучении тяготеет к примерам большего класса и ей не хватает обучающих примеров единичного класса, чтобы корректно вывести разделяющую функцию. Т.е. в данном случае число примеров разных классов несбалансированное, примеров одного класса существенно меньше другого. Возможно, что это приводит к не очень качественному обучению (невысоким значениям F1 или неустойчивости и/или невысоким значениям в кросс-валидации).
    Можно сбалансировать число примеров обоих классов, применив алгоритмы балансировки данных. Эти алгоритмы генерируют искусственные примеры меньшего класса на основе его фактических примеров из выборки, добиваясь того, чтобы число примеров различных классов было одинаковым.
    Попробуем применить балансировку данных к лучшему результату из предыдущих экспериментов и, докрутив параметры нейросети, посмотреть, как это влияет на качество обучения модели.
    Для балансировки данных можно воспользоваться алгоритмами SMOTE и ADASYN библиотеки imblearn:
    https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.SMOTE.html
    https://imbalanced-learn.org/stable/references/generated/imblearn.over_sampling.ADASYN.html

    Для этого необходимо добавить импорт из библиотеки и вызвать сам алгоритм балансировки:

```
from imblearn.over_sampling import SMOTE

Data = pd.read_excel('Data_Set_(A+B).xlsx')
// Out = ... - в соответствии с вариантом задания
// X = Data.drop(['Type', ... - в соответствии с вариантом], axis=1)

scaler = MinMaxScaler() # Инициализация нормализатора
```

```
X_Norm = scaler.fit_transform(X) # Нормализация и запись результата в X_Norm

# Балансировка данных
smote = SMOTE(random_state=42) # Инициализация алгоритма балансировки
X_bal, Y_bal = smote.fit_resample(X, Y) # Балансировка, результат в X_bal, Y_bal
...

ANN.fit(X_bal, Y_bal)

...
```

! Обратите внимание, что после получения балансированных значений в матрице X_bal, Y_bal последние используются в обучении сети вместо X и Y. Более того, далее они используются везде в рамках данного эксперимента (см. п.11) – и в ANN.predict, и в функции cross_val_score, и в выводе критериев Accuracy и F1, и в выводе графиков (необходимо сделать такую подстановку самостоятельно).

11. Провести эксперимент по п.3 с балансированными данными (необходимо применить оба алгоритма балансировки, в примере выше приведено использование алгоритма SMOTE, алгоритм ADASYN вызывается и используется абсолютно аналогично, необходимо лишь в импорте указать ADASYN вместо SMOTE, а также в строке с инициализацией алгоритма). При этом, в качестве выборки данных, которая загружается в переменную Data из Excel, необходимо выбрать ту (с нормализацией или без нормализации), на которой в предыдущих экспериментах результат получился лучше всего по совокупности кросс-валидации и критерию F1.

12. Зафиксировать лучший полученный результат по п.8 – выставить в блокноте результат лучшего найденного результата обучения (график и значения критериев понадобятся для отчёта).
    Добавить в блокноте новое поле ниже, выбрать вместо «Code», режим «Markdown» и написать заголовок:

    # Проверка сети на контрольной выборке C

13. ***Для лучшего результата обучения моделей*** (в рамках всех экспериментов) проводится проверка качества обучения на независимой выборке C. Необходимо выполнить операцию “predict” на выборке C, построить графики и вывести значения критериев Accuracy и F1 (кросс-валидация на контрольной выборке не проводится).

    <u>! Примечание</u>: При поверке качества модели на независимой выборке C операция обучения “fit” не должна применяться к данным выборки C.

Рекомендации к формированию отчёта по практикуму

Написание отчёта по данному практикуму является творческим процессом, поскольку работа помимо приобретения знаний и навыков в области ИИ направлена также и на развитие исследовательских компетенций.
К отчёту прилагается файл Jupyter Notebook в формате ipynb.

В обязательном порядке в отчёте приводятся:

- <u>графики сравнения табличного значения выходной величины и выхода сети для лучших результатов исследований по пунктам 4, 6, 9, 12</u> с указанными значениями критерия кросс-валидации, Accuracy и F1, а также параметрами нейросети;

- краткие основные выводы (по ходу выполнения практикума) по влиянию параметров сети на точность получаемых моделей;
- графики сравнения табличного значения выходной величины и выхода сети для случая проверки на выборке «С» с подписанными значениями настроек сети, а также указанными значениями Accuracy и F1;
- итоговые выводы по работе в целом, включая выводы по результатам предобработки данных: сортировки, нормализации и балансировки.

***! Примечание*:** Выше указаны обязательные для отчёта пункты. Однако, приветствуется приведение автором в отчёте дополнительных графиков/результатов, которые иллюстрируют некоторые важные/интересные промежуточные результаты выполнения работы, например, высокое качество обучения или, наоборот, демонстрируют плохую обучаемость сети при текущих параметрах настройки алгоритма, или иллюстрируют наличие точек-выбросов (outliers) в обучении и т.п. или какие-то другие важные или интересные моменты, наблюдаемые в процессе исследования.

Варианты заданий на лабораторную работу №2

№ вар.	Тип поверхностей	Использование переменных*	ФИО исполнителя
1	1	$\{V_1\}$	
2	2	$\{V_1\}$	
3	3	$\{V_1\}$	
4	4	$\{V_1\}$	
5	5	$\{V_1\}$	
6	1	$\{V_2\}$	
7	2	$\{V_2\}$	
8	3	$\{V_2\}$	
9	4	$\{V_2\}$	
10	5	$\{V_2\}$	
11	1	$\{V_1, V_2\}$	
12	2	$\{V_1, V_2\}$	
13	3	$\{V_1, V_2\}$	
14	4	$\{V_1, V_2\}$	
15	5	$\{V_1, V_2\}$	
16	1	$\{V_1, V_3\}$	
17	2	$\{V_1, V_3\}$	
18	3	$\{V_1, V_3\}$	
19	4	$\{V_1, V_3\}$	
20	5	$\{V_1, V_3\}$	
21	1	$\{V_2, V_3\}$	
22	2	$\{V_2, V_3\}$	
23	3	$\{V_2, V_3\}$	
24	4	$\{V_2, V_3\}$	
25	5	$\{V_2, V_3\}$	